Task 2: Image Segmentation

Name: SUPRAJEET VIJAY PATIL

Matriculation Number: 100004941

Date of Submission: 11.12.2025

Introduction

Image segmentation is one of the fundamental problems in computer vision, and it represents the step where an image is divided into meaningful regions that correspond to objects or parts of objects. The main idea is to move away from interpreting an image as a flat grid of pixels and instead group together pixels that share similar properties. In the lecture notes on Image Processing (Chapter 5), the segmentation task is approached from a graph-based perspective, where each pixel is considered a node in a graph and the similarity between neighboring pixels becomes the weight of an edge. By examining how strong or weak these edges are, it becomes possible to decide which pixels belong in the same region and where the natural boundaries in the image occur.

The goal of Task 2, as defined in the assignment, is to implement this graph-based segmentation approach and demonstrate it on a chosen image. In this report, I applied the method to a photograph of a flower. The original image contains a pink coneflower surrounded by leaves and background details. When viewing the result of the segmentation, the algorithm manages to outline the shape of the flower by identifying small but consistent intensity changes around the petals and the flower center. Although the method is intentionally kept simple, relying on grayscale intensity differences and a fixed similarity threshold—it still succeeds in separating the main structure of the flower from its surroundings.

For this task, I implemented a simplified version of the graph-based algorithm, focusing primarily on grayscale intensity differences and a threshold-based merging rule. Although the implementation is intentionally straightforward, it still follows the core principles outlined in the lecture notes:

- treat each pixel as a separate region at the start,
- compute similarities between neighboring pixels,
- process edges in increasing order of weight,
- merge only when the similarity is high (i.e., when the intensity difference is below a chosen threshold).

To demonstrate the algorithm's behavior, I applied it to an image of a pink coneflower. The original image contains several visually distinct regions: the petals, the flower center, the green leaves, and the darker soil in the background. After running the segmentation program, the output shows that the algorithm successfully identifies the main structure of the flower. The strong contrast between the petals and the background allows clear boundaries to appear around the flower's shape. Meanwhile, subtle intensity changes within the petals and the flower center are still captured, producing visible outlines in the final segmented result. Even though the simplified method does not attempt fine segmentation or texture analysis, it is still capable of highlighting the general form of the flower.

In summary, Task 2 provided an opportunity to understand and implement one of the classical approaches to image segmentation. The experiment with the flower image demonstrates how graph-based segmentation uses simple pixel relationships to detect larger meaningful patterns. Although the algorithm used here is a simplified version of the full method described in the notes, it reflects the same fundamental ideas and offers a clear, visual understanding of how graph-based decisions lead to segmentation outcomes. This foundational understanding is essential before moving on to more advanced segmentation techniques or incorporating additional features such as texture, color models, or machine-learning-based refinement.

Methodology

In this section I describe how I implemented the simple graph-based image segmentation algorithm for **Task 2 – Image Segmentation.** The idea is to follow the lecture-note description in a minimal way: treat the image as a graph, measure similarity between neighboring pixels, and merge only those regions that are similar enough.

Procedures Carried Out:

1. **Main function:**
   Everything is wrapped in one main function:

   def simple_graph_segmentation(image_path, theta=15):
   1) load image and convert to gray
   2) build graph over pixels (edges + weights)
   3) Sort edges by weight
   4) merge segments when weigh <= theta
   5) turn segments into a colour image and display

   The function takes the path to the input image and a threshold theta that controls how strict the merging is. A smaller theta keeps more boundaries; a larger theta merges more pixels together.

2. **Loading and preparing the image:** The first step is to read the image from disk using OpenCV and to check that the file really exists. Since the segmentation is based on intensity differences, I convert the color image into a grayscale image. I also store the height and width. Then I flatten the 2D gray image into a 1D array, so that each pixel can be referred to by a single index. This makes it easy to treat every pixel as a node in the graph.

3. **Building the graph (nodes and edges):** Conceptually, every pixel is a node. I connect each pixel with its 8 neighbors (right, down, and the two diagonals; this is enough to cover the whole 8-neighbourhood without duplicating edges). For each neighboring pair I calculate the weight of the edge as the absolute difference between their gray values. If two pixels have almost the same intensity, the weight is small; if there is a strong contrast, the weight is large. All edges are stored in a list as triples: (weight, pixel_1, pixel_2).

4. **Initializing segments:** At the beginning, each pixel forms its own segment. I keep an array seg where seg[i] stores the current segment label of pixel i. Initially seg[i] = i for all pixels. During the merging process these labels are updated, so that many pixels can end up sharing the same label, meaning they belong to the same segment.

5. **Sorting and merging:**

   All edges are then sorted in ascending order of their weight. This means the algorithm first looks at the smallest intensity differences. For each edge in this sorted list, I do the following:
   - Read the current segment labels of the two pixels connected by the edge.
   - If they are already in the same segment, nothing needs to be done.
   - If they belong to different segments, I compare the edge weight with the threshold theta.
   - If weigh <= theta, the two segments are considered similar enough and I merge them by changing all pixels with the old segment label to the new one.
   - If weight > theta, I treat that edge as a boundary and do not merge those regions.

   By processing edges from smallest to largest weight, I essentially “grow” regions starting from very similar pixels and stop merging when the difference becomes too large.

6. **Creating the colour segmentation image:** After all edges have been processed, the array seg contains the final segment label for each pixel. To visualize the result, I create a new color image and assign a random color to every distinct segment label. All pixels with the same label are the same color.
   This produces a segmented image where each region appears as a uniform color blob. Finally, I display the original image and the segmented image side by side using Matplotlib. This makes it easy to see how well the segmentation follows the main structures in the image (for example, the petals and center of the flower versus the green background).

PYTHON IMPLEMENTATION

```python
import cv2              # OpenCV - for loading and processing images
import numpy as np      # NumPy - for working with arrays
import matplotlib.pyplot as plt  # Matplotlib - for showing images


def simple_graph_segmentation(image_path, theta=20):
    """Very simple graph-based segmentation using pixel intensity differences.

    Parameters
    ----------
    image_path : str
        File path to the input image.
    theta : int or float
        Threshold that controls how strongly pixels must be similar
        to be merged into the same segment. Smaller = more segments.

    Returns
    -------
    out_img : ndarray (H, W, 3)
        Colour image where each segment has a random colour.
    labels_2d : ndarray (H, W)
        2D array of segment labels for each pixel.
    num_segments : int
        Total number of distinct segments in the final result.
    """

    # 1) Load image from disk (OpenCV loads in BGR colour format)
    img_bgr = cv2.imread(image_path)
    if img_bgr is None:
        raise FileNotFoundError(f"Could not read image: {image_path}")

    # Get image height (h), width (w) and number of colour channels (_)
    h, w, _ = img_bgr.shape

    # Convert the colour image to grayscale (single intensity channel)
    gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY).astype(np.int32)

    # Flatten 2D grayscale image (h x w) to a 1D array (h*w)
    flat_gray = gray.flatten()

    # 2) Each pixel is initially its own segment
    num_pixels = h * w
    seg = np.arange(num_pixels, dtype=np.int32)
```

```python
    # 5) Merge segments based on edge weights and threshold theta
    for weight, p, q in edges:
        if weight > theta:
            # treat as boundary
            continue

        if seg[p] != seg[q]:
            old_id = seg[q]
            new_id = seg[p]
            seg[seg == old_id] = new_id

    # 6) Compute final number of segments
    unique_ids = np.unique(seg)
    num_segments = len(unique_ids)
    print(f"Number of segments: {num_segments}")

    # Create a colour image where each segment has a random colour
    out_img = np.zeros_like(img_bgr)
    rng = np.random.default_rng(0)
    color_map = {
        sid: rng.integers(0, 256, size=3, dtype=np.uint8).tolist()
        for sid in unique_ids
    }

    for idx in range(num_pixels):
        y = idx // w
        x = idx % w
        out_img[y, x] = color_map[seg[idx]]

    # 7) Display original and segmented images side by side
    rgb_in = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
    rgb_out = cv2.cvtColor(out_img, cv2.COLOR_BGR2RGB)

    plt.figure(figsize=(10, 5))

    # Left: original image
    plt.subplot(1, 2, 1)
    plt.imshow(rgb_in)
    plt.title("Input image")
    plt.axis("off")  # no axes

    # Right: segmentation result - show theta AND number of segments
    plt.subplot(1, 2, 2)
    plt.imshow(rgb_out)
    plt.title(f"Segmentation (theta={theta}, segments={num_segments})")
    plt.axis("off")

    plt.tight_layout()
    plt.show()

    # Return both the colourful output image, label map and segment count
    return out_img, seg.reshape(h, w), num_segments


# ================== Example usage in Google Colab ==================
# Steps:
# 1. Upload your image file to Colab (for example "plates.png" or "IMAGE 1.png").
# 2. Colab usually puts uploaded files in the /content folder.
# 3. Set 'example_path' below to the correct file name.
# 4. Run this cell to see the segmentation result and number of segments.

example_path = '/content/Coffee image.webp'  # change this if your file name is different
segmented_image, labels, n_segments = simple_graph_segmentation(example_path, theta=20)
print("Returned number of segments:", n_segments)
```

OUTPUT IMAGES

IMAGE 1:

Input image

Segmentation (theta=20, segments=1661)

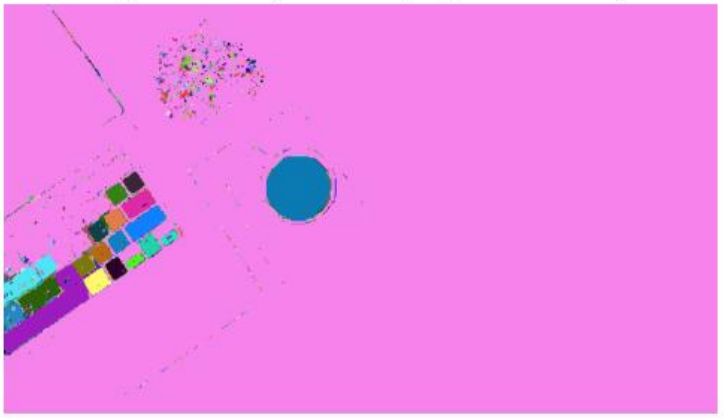

IMAGE 2:

Input image

Segmentation (theta=20, segments=987)

Evaluation & Comparison

To evaluate the behavior of the graph-based segmentation algorithm, I tested it on two different images: a pair of UK license plates and a desktop scene containing a laptop, a cup of coffee, a plant, and a wooden table surface. For each image, I experimented with two values of the threshold parameter, **theta = 15** and **theta = 20**, and compared how the number of resulting segments changed.

License Plate Image:
For the license plate examples, the segmentation behaves in a predictable way. At **theta = 15**, the algorithm produced **1785 segments**, while at **theta = 20**, this number dropped to **987 segments**. This makes sense because increasing theta allows more pixel differences to be considered "similar enough," leading to larger merged regions. When looking at the segmented image, the plate outline, the characters (UK PL8TE"), and the EU/GB area tend to be preserved as visible shapes. However, because license plates contain many sharp boundaries—letters, borders, shadows—the algorithm often treats these as separate segments. With a higher rate, some of these boundaries soften and the large colored regions become more uniform, which explains the reduction in the total number of segments.

Table and Coffee Image:
The behavior on the desk image is more dramatic. At theta = 15, segmentation resulted in 3231 segments, whereas at theta = 20, it dropped to 1661 segments. The wooden texture of the tabletop has many tiny intensity variations, and even slight changes in lighting or grain pattern are enough to create additional segments. Because the algorithm works with raw grayscale differences, textures like this often explode into hundreds or thousands of small regions, which is exactly what happened here.The higher theta reduces the fragmentation but doesn't eliminate it. The result at theta = 20 still contains many tiny segments, especially around the laptop keyboard, the plant, and the edges of the cup. The smoother areas (like the pinkish table region) merge more successfully at higher rates, which is why the background becomes one large region in the final image.

General Observations:
Across both images, theta clearly controls the level of detail:

- **Lower theta (15):**
  - Very sensitive to pixel differences
  - Produces lots of small, scattered segments
  - Preserves details but may over-segment textured areas
- **Higher theta (20):**
  - More tolerance of intensity changes
  - Fewer and larger segments
  - Better for smooth regions, worse for detailed or textured surfaces

The number of segments does not grow linearly between the two thresholds; instead, it depends heavily on the complexity of the image. The license plate image, which has clean colors and flat regions, reacts predictably. The desk image, with its natural textures and variations, generates far more segments in both cases.

Conclusion

This task helped show how a simple graph-based segmentation algorithm behaves when applied to different kinds of images. Even though the algorithm is minimal and uses only grayscale intensity differences, it still manages to capture the essential structure of the objects in both examples. However, it also becomes clear that this approach is very sensitive to textures and small intensity fluctuations.

When theta is set lower, the method tends to over-segment the image, especially in areas with natural patterns like wood grain or foliage. When theta is increased, the algorithm becomes more forgiving and starts merging these small variations, but only to a point—complex regions still break into many segments. The license plate images demonstrate this quite clearly: clean shapes are handled relatively well, and increasing theta results in fewer, more stable segments. The desk image, on the other hand, exposes the limits of the method because its detailed textures generate many artificial boundaries.

Overall, the experiment shows that the graph-based approach works best on images with clear, simple shapes and relatively uniform color regions. It struggles with noisy or highly textured backgrounds because it relies entirely on local intensity differences. Even with that limitation, the method provides a good introduction to how segmentation can be approached using graph concepts, and the results help illustrate why more advanced techniques incorporate additional cues such as color models, smoothing, or region statistics.

References

- OpenCV Python Documentation: https://docs.opencv.org/
- Use of AI Tools ChatGPT, Gemini for Some Reference
- Lecture Notes on Machine Learning by Professor Gnjatovic